

SUPPORT DE COURS • DÉVELOPPEMENT ANDROID

# Kotlin & Jetpack Compose

Le langage moderne de Google et son toolkit d'interface déclarative pour construire des applications Android natives.

## CE SUPPORT COUVRE :

- Fondamentaux Kotlin
- Programmation orientée objet
- Jetpack Compose
- État & mise en page
- Bonnes pratiques

# Sommaire

- 01** **Introduction à Kotlin**  
Origines, philosophie du langage et atouts pour Android.
- 02** **Syntaxe de base**  
Variables, fonctions, structures de contrôle et null safety.
- 03** **Programmation orientée objet**  
Classes, data class, objects, sealed class.
- 04** **Introduction à Jetpack Compose**  
UI déclarative, fonctions @Composable et recomposition.
- 05** **État et mise en page**  
remember, mutableStateOf, Column / Row / Box, Modifier.
- 06** **Bonnes pratiques & ressources**  
State hoisting, Preview, thème Material, liens utiles.

## À propos de ce support

Ce document propose une introduction condensée à Kotlin, le langage officiel du développement Android, puis à Jetpack Compose, le framework déclaratif recommandé par Google pour construire les interfaces utilisateur. Chaque chapitre associe explications concises et exemples de code directement réutilisables.

# Introduction à Kotlin

Kotlin est un langage de programmation moderne, statiquement typé, développé par JetBrains. Depuis 2019, Google en fait le langage privilégié pour le développement Android, aux côtés de Java avec lequel il est 100 % interopérable.

## Pourquoi Kotlin ?

- **Concis** — réduit le code répétitif (boilerplate) par rapport à Java.
- **Sûr** — le système de types distingue les valeurs nullable et non nullable, ce qui élimine une grande partie des *NullPointerException*.
- **Interopérable** — s'utilise dans un projet Java existant sans réécriture complète.
- **Moderne** — coroutines pour l'asynchrone, fonctions d'extension, expressions *when*.
- **Multiplateforme** — Kotlin Multiplatform permet de partager du code entre Android, iOS, le web et le serveur.

## Premier programme

```
fun main() {  
    val langage = "Kotlin"  
    println("Bonjour depuis $langage !")  
}
```

Le mot-clé `fun` déclare une fonction ; `val` déclare une valeur immuable.

## Où s'exécute Kotlin ?

- Applications Android (via Android Studio).
- Backend avec des frameworks comme Ktor ou Spring.
- Kotlin/JS pour le front-end web, Kotlin/Native pour du code natif.

# Syntaxe de base

## Variables : val vs var

```
val nom: String = "Ada" // immuable, ne peut plus changer
var age: Int = 30 // modifiable
age = 31 // OK
val pi = 3.14 // type déduit automatiquement
```

## Fonctions et structures de contrôle

```
fun addition(a: Int, b: Int): Int = a + b

fun decrire(note: Int): String = when {
    note >= 16 -> "Très bien"
    note >= 10 -> "Admis"
    else -> "Insuffisant"
}
```

En Kotlin, `if` et `when` sont des expressions : ils renvoient directement une valeur.

## La null safety

Un type est non nullable par défaut. Pour autoriser `null`, on ajoute un `?` après le type.

```
var email: String? = null // type nullable
val longueur = email?.length // appel sécurisé -> null si email est null
val taille = email?.length ?: 0 // valeur par défaut avec l'opérateur Elvis
```

# Programmation orientée objet

Kotlin conserve les concepts de la POO classique (classes, héritage, interfaces) tout en ajoutant des constructions qui réduisent fortement le code nécessaire.

## Classes et constructeur primaire

```
class Utilisateur(val nom: String, var age: Int) {  
    fun sePresenter() = "Je m'appelle $nom, j'ai $age ans"  
}  
  
val u = Utilisateur("Léa", 27)
```

## Data class

Une `data class` génère automatiquement `equals()`, `hashCode()`, `toString()` et `copy()`.

```
data class Produit(val nom: String, val prix: Double)  
  
val p1 = Produit("Clavier", 49.90)  
val p2 = p1.copy(prix = 39.90)
```

## Object et sealed class

```
object Config { // singleton  
    const val VERSION = "1.0"  
}  
  
sealed class Resultat  
data class Succes(val donnees: String) : Resultat()  
data class Erreur(val message: String) : Resultat()
```

Une `sealed class` restreint les sous-types possibles, ce qui permet au compilateur de vérifier qu'un `when` couvre bien tous les cas.

# Introduction à Jetpack Compose

Jetpack Compose est le toolkit moderne de Google pour construire des interfaces Android en Kotlin, sans XML. L'interface est décrite de façon **déclarative** : on décrit à quoi elle doit ressembler pour un état donné, et Compose se charge de mettre l'écran à jour.

## Les fonctions @Composable

Une interface se construit à partir de petites fonctions annotées `@Composable`, que l'on peut combiner comme des blocs Lego.

```
@Composable
fun Salutation(nom: String) {
    Text(text = "Bonjour, $nom !")
}

@Preview
@Composable
fun AperçuSalutation() {
    Salutation("Kotlin")
}
```

## Déclaratif vs impératif

- **Vues XML (impératif)** — on modifie manuellement chaque vue existante (`findViewById`, `setText...`).
- **Compose (déclaratif)** — on décrit l'UI en fonction de l'état ; lorsqu'un état change, Compose recalcule automatiquement ce qui doit être redessiné : la **recomposition**.

Compose s'appuie entièrement sur Kotlin : lambdas, fonctions de haut niveau et coroutines sont utilisées pour construire et animer l'interface.

# État et mise en page

## Gérer l'état avec remember / mutableStateOf

Compose relit une fonction @Composable lorsqu'un état observé change. `remember` conserve une valeur entre les recompositions, et `mutableStateOf` crée un état observable.

```
@Composable
fun Compteur() {
    var compte by remember { mutableStateOf(0) }

    Column {
        Text(text = "Valeur : $compte")
        Button(onClick = { compte++ }) {
            Text("Incrémenter")
        }
    }
}
```

## Organiser l'écran : les layouts

- `Column` — empile les éléments verticalement.
- `Row` — aligne les éléments horizontalement.
- `Box` — superpose les éléments (utile pour les badges, overlays...).

## Le Modifier

Le paramètre `Modifier` permet de chaîner taille, marges, clics ou styles sur n'importe quel composant.

```
Text(
    text = "Bienvenue",
    modifier = Modifier
        .padding(16.dp)
        .fillMaxWidth()
)
```

# Bonnes pratiques & ressources

## Bonnes pratiques Compose

- **State hoisting** — remonter l'état au composant parent pour garder les composants enfants simples et réutilisables (« stateless »).
- **Petits composables** — préférer plusieurs fonctions courtes et ciblées à une seule fonction géante.
- **@Preview** — visualiser un composant dans Android Studio sans lancer l'application.
- **MaterialTheme** — centraliser couleurs, typographies et formes pour une UI cohérente.
- **Éviter la logique métier dans l'UI** — la déléguer à un ViewModel.

## Bonnes pratiques Kotlin

- Préférer **val** à **var** autant que possible.
- Utiliser les *data class* pour représenter des données simples.
- Exploiter les fonctions d'extension pour enrichir des classes existantes sans les modifier.
- Gérer l'asynchrone avec les coroutines plutôt qu'avec des callbacks imbriqués.

## Pour aller plus loin

|                                                           |                                                                       |
|-----------------------------------------------------------|-----------------------------------------------------------------------|
| <b>Documentation Kotlin</b><br>kotlinlang.org             | <b>Documentation Compose</b><br>developer.android.com/jetpack/compose |
| <b>Codelabs Android</b><br>developer.android.com/codelabs | <b>Kotlin Playground</b><br>play.kotlinlang.org                       |

Kotlin et Jetpack Compose forment aujourd'hui la base recommandée par Google pour développer des applications Android modernes, robustes et rapides à faire évoluer. Bonne pratique !